Home Servant — Full App Documentation

This document describes, screen by screen, how the Home Servant Flutter prototype works today: every button, field, toggle, and the navigation between screens. It is written from the actual source code in `lib/`, quoting real screen titles and button labels as they appear there.

Table of Contents

---

1. Overview

Home Servant is a property-rental app with a bundled goods marketplace, built for three user roles:

- **Tenant** — browses and rents/books properties, manages a wishlist, messages landlords, and can also shop the Marketplace.
- **Landlord** — lists/manages their own properties (a simplified read-only tile list in this prototype) and can also enter the Marketplace.
- **Vendor** — a separate identity reached *through* the Marketplace (not through the main Login/Sign Up screens) that lists and sells physical goods (furniture, appliances, fittings, etc.) to tenants/landlords.

This is a UI prototype with no backend. Every screen's data is either hardcoded (`mockProperties`, `mockMarketplaceProducts`, seeded chat threads/notifications) or held in plain in-memory/local-storage state:

- `lib/state/app_state.dart` is a single `ChangeNotifier` (`AppState`) that holds the entire signed-in session — profile fields, wishlist, rental history, notification toggles, 2FA/App-Lock settings, theme — and is only persisted locally on-device via `SharedPreferences` (so it survives an app restart, but nothing is ever sent to a server).
- **Login accepts any credentials.** `LoginTenantScreen` / `LoginLandlordScreen` and `VendorLoginScreen` never check the username/password against anything — tapping **Login** (after a non-empty check on the vendor side) always proceeds.
- **OTP/2FA accepts any 4-digit code.** `VerifyOtpScreen` only checks that 4 digits were entered, not that they match anything a backend issued.
- **Payments are simulated.** The Marketplace checkout modal has real payment-method choices (Card, Bank Transfer, Pay on Delivery) but tapping "Pay ₦…" always immediately succeeds and records the order locally — there is no payment gateway.
- **Vendor identity is a single mock shop.** `models/vendor.dart` notes explicitly: "This prototype has no vendor auth backend — logging in with any email/password lands on this one demo shop." Signing up as a new vendor shows a submission-received screen but doesn't actually create a new logged-in shop.
- Deactivating/deleting an account or a vendor shop just clears the local session; there is no server record to change.

Tech stack:

- **Flutter** (Material 3), with a phone-first design that gets width-clamped (`ResponsiveCenter`, `gridColumnsForWidth`) rather than redesigned for tablet/web.
- **go_router** for the *primary* auth → dashboard flow (`lib/routes/app_router.dart`) — a small, mostly linear route table.
- **Provider** (`ChangeNotifierProvider`) exposes the single global `AppState` to the widget tree from `lib/app.dart`.
- Almost everything *past* the dashboard (property detail, wishlist, settings, the entire Marketplace, vendor screens) is reached via plain `Navigator.push(MaterialPageRoute(...))` rather than go_router — these are nested pushes on top of the `/dashboard` route, not named routes.
- **Theming**: two parallel systems —
  - `UserRole` (`lib/models/user_role.dart`) drives the navy-vs-gold look of every onboarding/auth screen.
  - `DashboardTheme` (`lib/models/dashboard_theme.dart`) is a user-selectable palette (Midnight / Sand / Classic White) applied to the dashboard, marketplace, and vendor screens once signed in, chosen from the profile's **Theme** row.
- Fonts: **Quity** for headings, **Givonic** for body text (`AppTextStyles`).

---

2. Navigation Map

`lib/routes/app_router.dart` defines the following `GoRoute`s, all on one flat list (no nested routers):

- `/` — `SplashScreen`. Shows a Ken-Burns-animated background photo (with an optional looping video overlay once it loads) and an **EXPLORE** button → pushes `/get-started`.
- `/get-started` — `GetStartedScreen` ("Get Started with Home Servant"). Two buttons: **Login** → pushes `/login`; **Sign Up** → pushes `/signup`.
- `/login` — `LoginScreen` ("Welcome Back — Tell us which side of Home Servant you're on."). Two buttons: **Login as a Tenant** and **Login as a Landlord**. Either sets the chosen role on `AppState` and pushes `/login-tenant` or `/login-landlord`.
- `/login-tenant` — `LoginTenantScreen`. Username/Password fields + **Login** button → runs `_proceedAfterLogin`. A "Don't have an account? Sign Up" link pushes `/signup`.
- `/login-landlord` — `LoginLandlordScreen`. Same shape, "Sign Up" link pushes `/signup-landlord`.

    - `_proceedAfterLogin` : if `AppState.twoFactorEnabled` is true, goes to `/login-2fa` ; otherwise calls `_proceedPastTwoFactor` directly.
    - `_proceedPastTwoFactor` : if `AppState.appLockEnabled` and a PIN is set, goes to `/app-lock-verify` ; otherwise goes straight to `/dashboard` .
- `/signup` — `SignupScreen` ("Create an Account — Tell us which side of Home Servant you're on."). **Sign up as a Tenant** / **Sign up as a Landlord** → sets role, pushes `/signup-tenant` or `/signup-landlord` .
- `/signup-tenant` / `/signup-landlord` — email-collection screens ("Sign Up — Enter your email to signUp"). An email field with validation (must contain `@`), a **Continue** button, a "Sign in with social media" divider, and a **Continue with Google** button (which — since there's no real OAuth — just calls the same `onContinue(email)` callback with whatever's in the field). Both push `/verify-otp` after setting `AppState.email` .
- `/verify-otp` — `VerifyOtpScreen` ("Verify Your Email"). A 4-digit OTP box row, a 50-second countdown, **Continue** (enabled only once 4 digits are entered), and **Resend OTP** (enabled only once the timer hits 0). On verify, branches by role: landlord → `/signup-landlord-1` ; tenant → `/signup-tenant-1` .
- `/signup-landlord-1` → `SignupLandlord1Screen` (name, phone, house address, **Certificate of Ownership** upload button) → `/signup-landlord-2` .
- `/signup-landlord-2` → `SignupLandlord2Screen` (photo, document upload, means-of-identification picker, ID number field) → **Continue** calls `onFinish` → `context.go('/dashboard')` .
- `/signup-tenant-1` → `SignupTenant1Screen` (name, phone, optional referral code) → `/signup-tenant-2` .
- `/signup-tenant-2` → `SignupTenant2Screen` (photo, address, means-of-identification picker, ID number field) → **Continue** calls `onFinish` → `context.go('/dashboard')` .
- `/login-2fa` — reuses `VerifyOtpScreen` , wired so `onVerified` calls `_proceedPastTwoFactor` instead of continuing signup.
- `/app-lock-verify` — `AppLockScreen` with the stored PIN; `onUnlocked` goes to `/dashboard` . (If somehow reached with no PIN set, it silently redirects to `/dashboard` instead of crashing.)
- `/dashboard` — builds `LandlordDashboardScreen` if `AppState.role.isLandlord` , otherwise `TenantDashboardScreen` .

Everything below `/dashboard` — property detail/gallery, wishlist, messages, notifications, profile, settings, history, legal docs, the entire Marketplace (customer and vendor sides) — is reached by `Navigator.of(context).push(MaterialPageRoute(...))` calls from within these two dashboard screens, not by further go_router routes. The Marketplace in particular is entered by tapping the second bottom-nav icon (cart icon) on either dashboard, which pushes `MarketplaceAuthScreen` on top of the current route.

```
/  (Splash)
 └─ EXPLORE → /get-started
     ├─ Login → /login → role choice → /login-tenant | /login-landlord
     │     └─ Login → [2FA?] → [App Lock?] → /dashboard
     └─ Sign Up → /signup → role choice → /signup-tenant | /signup-landlord
           └─ email+Continue → /verify-otp → role branch
                 ├─ /signup-tenant-1 → /signup-tenant-2 → /dashboard
                 └─ /signup-landlord-1 → /signup-landlord-2 → /dashboard

/dashboard (role-gated)
 ├─ TenantDashboardScreen — bottom nav: Home | Marketplace | (description icon, unused) | Profile
 └─ LandlordDashboardScreen — same bottom nav shape, "My Properties" list instead of a feed
       both: tapping the 2nd nav icon pushes MarketplaceAuthScreen (Navigator, not go_router)
```

---

3. Onboarding & Authentication

Splash Screen (`lib/features/splash/splash_screen.dart`)

Full-bleed background photo (`homepage.jpg`) with a subtle Ken-Burns zoom/pan animation, a gradient overlay, the Home Servant logo, the tagline "Find Your Perfect House Just one Click Away", and an **EXPLORE** button. On non-web platforms it also tries to load and play a looping background video (`assets/videos/splash_bg.mp4`), falling back silently to the still photo if it fails to load within 8 seconds. EXPLORE pushes `/get-started` .

Get Started (`lib/features/onboarding/get_started_screen.dart`)

Title "Get Started with Home Servant", subtitle "Find your dream home or manage your properties with ease." Two buttons: **Login** and **Sign Up** (plus a back arrow if there's a page to pop to).

Role selection — Login (`lib/features/auth/login_screen.dart`)

"Welcome Back — Tell us which side of Home Servant you're on." Buttons: **Login as a Tenant**, **Login as a Landlord**.

Role selection — Sign Up (`lib/features/auth/signup_screen.dart`)

"Create an Account — Tell us which side of Home Servant you're on." Buttons: **Sign up as a Tenant**, **Sign up as a Landlord**.

Tenant Login (`login_tenant_screen.dart`) / Landlord Login (`login_landlord_screen.dart`)

Identical layout, tenant uses navy theme, landlord uses gold/sand theme (via `UserRole`). Fields: **Username**, **Password** (obscured). Button: **Login** — no validation at all; any input (even empty) proceeds through `_proceedAfterLogin` . Link: "Don't have an account? Sign Up". Footer: `TermsFooter` ("By clicking continue, you agree to our Terms of Service and Privacy Policy").

Tenant Sign Up (`signup_tenant_screen.dart`) / Landlord Sign Up (`signup_landlord_screen.dart`)

Title "Sign Up", subtitle "Enter your email to signUp". One field: email (validated to contain `@`). Button: **Continue**. Divider "Sign in with social media" and a **Continue with Google** button (`GoogleSignInButton`) that, since there's no real Google auth wired up, just calls the same continue callback with the typed email.

Verify OTP (`verify_otp_screen.dart`) — used at signup *and* reused for login-time 2FA

Title "Verify Your Email — Enter the OTP sent to your Email Account". A 4-box digit entry (`OtpInputRow` , auto-advances focus between boxes) and a live "0:SS" countdown starting at 50 seconds. **Continue** is disabled until all 4 digits are entered — any 4 digits are accepted, there's no real code check. **Resend OTP** is greyed out and reads "Resend available after timer ends" until the countdown reaches 0, then becomes tappable and restarts the timer.

Tenant Sign Up Step 1 (`signup_tenant1_screen.dart`)

Title "Welcome Onboard". Fields: **Enter your Name**, **Phone Number**, **Referral code (optional)**. Button: **Continue**, which hands the three field values back via a callback (no validation).

Tenant Sign Up Step 2 (`signup_tenant2_screen.dart`)

A tappable photo panel ("Add a Photo") using the shared upload picker. Address field (multi-line, "Enter your address"). A **Means of Identification** dropdown-style tile that opens a bottom sheet listing: **NIN**, **Driver's License**, **Voter's Card**, **International Passport**. Choosing one reveals an ID-number field whose hint/format/length is tailored per document type (e.g. NIN: 11-digit numeric; Voter's Card: 19-character VIN, uppercased). Button: **Continue** → calls `onFinish` (no validation is actually enforced on submit).

Landlord Sign Up Step 1 (`signup_landlord1_screen.dart`)

Title "Welcome Onboard". Fields: **Enter your Name**, **Phone Number**, **House Address**. An outline button: **Certificate of Ownership** (opens the upload picker; its label swaps to the picked file's name). Button: **Continue**.

Landlord Sign Up Step 2 (`signup_landlord2_screen.dart`)

Same shape as tenant step 2 but landlord-themed: photo panel, an **Upload your document** outline button, the same Means of Identification bottom sheet + ID-number field logic, and **Continue** → `onFinish` .

All auth/onboarding screens share `ThemedScaffold` (solid navy or gold/sand background, scrollable body, back arrow when there's a page to pop) and `TermsFooter` .

---

4. Tenant Dashboard & Related Screens

Tenant Dashboard (`lib/features/dashboard/tenant_dashboard_screen.dart`)

The main shopping-for-a-home feed. Structure:

- **Top bar**: a location pin + "Ikeja, Lagos" (static, not derived from GPS), a mail icon (pushes `MessagesScreen`), and a bell icon (pushes `NotificationsScreen` ; a red dot shows while `_hasUnreadNotifications` is true and clears the moment the bell is tapped).
- **Search bar**: "Search by location or property" — filters live against property title/location; a clear (×) button appears once text is entered.
- **Filter button** (tune icon) next to the search bar opens the **Filters** bottom sheet:
  - **Price Range** — two numeric fields (Min/Max, thousands-separated, prefixed with "₦") plus a `RangeSlider` , bounded by the min/max price actually present in the *currently selected category* (so switching category resets any price filter, since yearly-rent and nightly-shortlet prices aren't comparable).
  - **Sort by Price** — choice chips: **None**, **Low to High**, **High to Low**.
  - **State** — a dropdown of all 36 Nigerian states + FCT (`nigerianStates`); all seed listings are in Lagos, so picking any other state legitimately shows zero results.
  - **Location in [State]** — appears only once a state is picked; free-text sub-filter (e.g. "Ikeja, Lekki, Yaba…").
  - **Apply** button commits the sheet's local state back into the screen's filter state and closes the sheet.
- **Categories** row: **House**, **Shortlet**, **Self-Con**, **Apartment** — tapping one filters the feed and resets price/sort filters (not the state/location filter).
- **Property grid/list**: `PropertyCard` s (1 column on phone, up to 3 on wide/desktop via `gridColumnsForWidth`), each showing the photo, a heart/favorite toggle (writes to `AppState.favoritePropertyIds`), a 4-dot carousel indicator (decorative, doesn't page), title, location, and a 2-star rating snippet. Tapping a card opens `PropertyDetailScreen` .
- **Bottom nav** (`DashboardBottomNav` , 4 icons: home / cart / description / person): tapping index 0 shows the feed, index 1 pushes `MarketplaceAuthScreen` , index 3 swaps the body for `ProfileScreen` in place (no navigation — it's a local `_navIndex` state flip inside the same `Scaffold`). Index 2 ("description") is present in the icon set but has no distinct screen wired to it in this build — the body only special-cases index 3 (Profile), so tapping index 2 just sets `_navIndex = 2` and the home feed stays on screen exactly as it was, making it a visual no-op.

Property Detail (`property_detail_screen.dart`)

A large hero photo (tap to open the gallery) with back and favorite-heart buttons overlaid, followed by a rounded sheet containing:

- Feature icons: bedrooms count, bathrooms count, "Kitchen".
- Title, location, star rating, and price label (e.g. "₦350,000/year").
- An expandable **Description** ("Read more" / "Read less", clamped to 3 lines when collapsed).

- A **Details Preview** strip of thumbnail interior photos (kitchen/bathroom/bedroom) — tapping one opens the gallery at that photo.
- **Rent Now** / **Book Now** button (label depends on category: "Book Now" for Shortlet, "Rent Now" otherwise) — calls `AppState.recordRentalOrBooking(propertyId, isShortlet: ...)`, which writes/overwrites a `RentalRecord` starting today (1 year for a rent, 3 days for a shortlet booking), and shows a snackbar ("Rent request sent for …" / "Booking request sent for …").
- **Message Landlord** / **Message Owner** outline button (label also depends on category) — opens `ChatThreadScreen` pre-seeded with an opening message from the tenant asking if the property is still available.

Property Gallery (`property_gallery_screen.dart`)

Full-screen black `PageView` of the hero photo + gallery images, pinch-zoomable (`InteractiveViewer`), an app-bar counter ("2/4 · Property Title"), and a row of dot indicators at the bottom.

Wishlist (`wishlist_screen.dart`)

Title "WishList". Lists every `mockProperties` entry whose id is in `AppState.favoritePropertyIds`, rendered as the same `PropertyCard`s used on the dashboard. Empty state: a heart icon, "No favorites yet", "Tap the heart on any property to save it here for later."

Messages (`messages_screen.dart`) + Chat Thread (`chat_thread_screen.dart`)

`MessagesScreen` lists 3 seeded conversations (two landlords, one "HomeServant Support"), each with a name, preview line, timestamp, and an unread dot that clears once the thread is opened. Tapping a row pushes `ChatThreadScreen` with that conversation's `initialMessages`. The chat screen is a simple bubble list (sent messages right-aligned in the theme's accent color, received left-aligned) with a text field and a send button; sent messages are appended locally and never receive a scripted reply.

Notifications (`notifications_screen.dart`)

Title "Notifications". A static list of 4 seeded `NotificationItem`s (application approved, rent reminder, new message, new listing near you), each with an icon, title, body, and relative timestamp. Purely read-only — no per-item interaction.

Profile (`profile_screen.dart`)

Rendered *in place* of the dashboard feed when the Profile tab is selected (not a separate pushed route). Shows the avatar (or a placeholder person icon), an **Edit Profile** pill button, then a menu list:

- **WishList** → pushes `WishlistScreen`.
- **History** → pushes `HistoryScreen`.
- **Settings** → pushes `SettingsScreen`.
- **Theme** → opens the theme-picker bottom sheet; picking a theme calls `AppState.setDashboardTheme` (which also tries to swap the OS home-screen icon via `AppIconService`).
- **Invite Friends** → opens the invite-friends bottom sheet.
- **Log Out** → calls the `onLogOut` callback, which routes back to `/get-started` (session data itself is *not* cleared on a plain logout — only Deactivate/Delete does that).

Edit Profile (`edit_profile_screen.dart`)

Title "Edit Profile". A circular avatar with a camera-badge overlay (tap to pick a new photo) and a **Change Photo** text button. Each field — **First Name**, **Last Name**, **Date of Birth**, **House Address**, **Phone Number**, **Email**, **Password** — starts read-only (grey fill) and only becomes editable (white fill) after tapping its own pencil icon, which flips to a checkmark; this prevents an accidental tap from editing a saved value. Date of Birth is always read-only text but, once "editable," tapping it opens a native date picker instead of a keyboard. Password has a show/hide eye icon. Every field is pre-filled from `AppState` if set, otherwise from hardcoded dummy values (Jane Doe / jane.doe@example.com / etc.) so the form never looks empty. **Save Changes** validates the email (must contain `@`) and writes everything back to `AppState.updateProfileDetails`, shows a "Profile updated" snackbar, and pops.

Settings (`settings_screen.dart`)

Title "Settings", organized into four cards:

- **Account & Security**: **Change Password** (opens a bottom sheet with Current/New/Confirm Password fields; validates the current password against `AppState.password` if one is set, requires the new password to be ≥6 characters and to match its confirmation, then saves and shows "Password updated"). **Two-Factor Authentication** switch — turning it *on* pushes `VerifyOtpScreen` and only sets `twoFactorEnabled = true` once a 4-digit code is "verified"; turning it *off* asks for confirmation via a sheet ("Turn off Two-Factor Authentication? Your account will only be protected by your password."). **App Lock** switch — turning it on opens the Set-PIN sheet (`showSetAppLockPinSheet`) and only enables once a PIN is chosen and confirmed; turning it off requires re-entering the existing PIN via `showVerifyAppLockPinSheet` (or, if somehow no PIN is stored, disables immediately).
- **Notifications**: **Push Notifications** master switch; when off, four dependent switches (**New Messages**, **Property Updates**, **Wishlist Price Drops** — "Get notified when a saved property gets cheaper" — and **Promotions & Offers**) are visually dimmed and disabled (`IgnorePointer`) but keep their individually-stored values.
- **Support & Legal**: **Help & Support** → opens the shared support-options sheet (Call Support / Live Chat). **Privacy Policy** and **Terms of Service** → each pushes `LegalDocumentScreen` with the matching `LegalDocumentKind`.
- **Danger Zone** (red section header): **Deactivate Account** → confirmation sheet ("Your profile will be hidden and you'll be signed out. You can reactivate any time by logging back in.") → calls `AppState.deactivateAccount()` (clears the whole local session) then the `onAccountClosed` callback (routes to `/get-started`). **Delete Account** → confirmation sheet ("This can't be undone…") → same effect as deactivate in this prototype (`deleteAccount()` is currently identical to `deactivateAccount()` under the hood — both just clear the local session, since there's no backend record to actually delete).

History (`history_screen.dart`)

Title "History". Lists every property with a `RentalRecord` in `AppState.rentalHistory`, most recent start date first. Each tile shows the photo, title, location, a formatted date range ("Rented 4 Sep 2026 – 4 Sep 2027" or "Booked … – …" for shortlets), an **Active**/**Expired** status badge (green/grey, based on whether `endDate` is in the future), a 5-star rating row the tenant can tap to rate 1–5 (`AppState.rateHistoryProperty`), and a **Renew** (for a yearly rent) / **Rebook** (for a shortlet) button that calls `recordRentalOrBooking` again — resetting the period to start today while preserving any existing rating — and shows a "Renewed …" / "Rebooked …" snackbar. Empty state: "No history yet — Properties you rent or book will show up here."

Privacy Policy & Terms of Service (`privacy&terms_screen.dart`)

One `LegalDocumentScreen` widget serving both documents via `LegalDocumentKind.privacyPolicy` / `.termsOfService`, each rendering a small set of static, plain-language sections (e.g. Privacy: "What we store", "What we don't do", "Your controls"; Terms: "Using Home Servant", "Accounts", "Listings") — explicitly prototype-flavored copy, e.g. noting property photos are sourced from Unsplash for demonstration.

Shared dashboard widgets

- `DashboardBottomNav` (`widgets/bottom_nav.dart`) — the floating 4-icon pill nav (home/cart/description/person), themed via `DashboardTheme`.
- `PropertyCard` (`widgets/property_card.dart`) — the reusable listing tile with favorite-heart and star rating used by the feed, Wishlist, and History-adjacent detail links.
- `PropertyImage` (`widgets/property_image.dart`) — wraps `Image.asset` with a fallback to `homepage.jpg` if a property's photo path is ever missing.

---

5. Landlord Dashboard

`lib/features/landlord/landlord_dashboard_screen.dart` is deliberately much simpler than the tenant dashboard — there is no filter sheet, search, or category tabs. It shows:

- A top row with the Home Servant logo (icon-only, tinted to the current theme) and a notification bell (static red dot, no tap handler wired to open a notifications screen for landlords).
- A **"My Properties"** header with an **Add** pill button (navy background, gold "+ Add" text) — present in the UI but has no `onPressed` handler, so tapping it currently does nothing.
- A list of `_LandlordPropertyTile`s built from the *same* `mockProperties` list the tenant dashboard reads (so a landlord currently sees every mock property in the app, not just "their own" — there's no per-landlord filtering in this build). Each tile shows the photo, title, location, and a static "Active" status pill; tiles have no tap action.
- The same floating `DashboardBottomNav` as the tenant dashboard: index 0 shows this property list, index 1 pushes `MarketplaceAuthScreen` (identical entry point to the tenant side), index 3 swaps in the shared `ProfileScreen` (Wishlist/History/Settings/Theme/Invite Friends/Log Out — the same menu tenants get, unfiltered by role).

---

6. Marketplace — Customer Side

Marketplace entry (`marketplace_auth_screen.dart`)

Reached by tapping the cart icon in either dashboard's bottom nav. Title "Home Servant Marketplace", subtitle "Furniture, appliances, fittings and more — everything you need to move in, in one place." Three options:

- **Proceed as a Customer** → pushes `MarketplaceHomeScreen`.
- **Become a Vendor** → pushes `VendorSignupScreen`.
- **Already a vendor? Login as a Vendor** (text link) → pushes `VendorLoginScreen`.

Marketplace Home (`marketplace_home_screen.dart`)

Title "Marketplace" with three app-bar actions: a chat bubble icon (→ `MarketplaceMessagesScreen`), a receipt icon (→ `OrderHistoryScreen`), and a cart icon with a live item-count badge (→ opens the cart bottom sheet).

- **Search** field: "Search products or vendors" — matches product name or vendor name.
- **Category chips**: All, Furniture, Home Appliances, Electronics, Fittings & Fixtures, Décor, Tools & Equipment.
- **Product grid**: `MarketplaceProductCard`s showing photo (or a fallback icon tile), name, vendor name, star rating, price, a **stock indicator** (a green "`N` in stock" or red "Out of stock", driven by `product.stock`), a **quantity stepper** (–/+, floor 1, capped at `product.stock`; the + button disables itself at the cap), and a small circular **add-to-cart** button next to it. The button and its icon change state with the cart: normal (accent-colored cart icon) while the stepper's quantity isn't yet in the cart, and **green with a checkmark** once it is — tapping it again at that point doesn't re-add the item, it shows "Item already added to cart" instead. Once a stock-0 product exists the button also disables and greys out (relabeled nowhere here, since the card has no text label — the color/disabled state is the only signal). Whenever the cart already holds some quantity, a small "In cart: `N`" caption shows under the stepper. Tapping the card body (not the stepper/button) opens `MarketplaceProductDetailScreen`.
- **Cart** (bottom sheet, `_openCart`): lists each cart line with its photo, name, subtotal, a delete icon, its own **quantity stepper** (–/+, same floor-1/stock-cap rules, updates the running total live), and — only when a product supports more than one fulfillment method — fulfillment chips (**Delivery** / **Pickup**) to choose per item. Shows the running **Total** and a **Checkout** button.
- **Checkout → Payment modal** (`_openPaymentModal`): shows "Amount to pay: ~~₦~~…" and three selectable payment options — **Debit/Credit Card**, **Bank Transfer**, **Pay on Delivery** — then a **Pay ~~₦~~…** button.

Cart → Checkout → Order flow, end to end:

1. Adding to cart (from the home grid or the product detail screen) stores `productId → quantity` — always clamped to `1..product.stock` — and defaults each item's `FulfillmentMethod` to the first option that product supports (`MarketplaceProduct.fulfillmentOptions` , set by whichever vendor listed it). Tapping add-to-cart again with the *same* quantity already in the cart is a no-op that just surfaces the "Item already added to cart" snackbar rather than double-adding; picking a different quantity first (via either stepper) and tapping again updates the cart to that new quantity instead.
2. In the cart sheet, the shopper can change the quantity of any line directly with its stepper, or switch its `FulfillmentMethod` per item if the product supports both delivery and pickup.
3. Tapping **Pay ₦…** calls `_placeOrder` , which builds a `MarketplaceOrder` (id derived from a timestamp, `date: DateTime.now()` , the chosen `PaymentMethod` , the customer's name/phone/address pulled straight from `AppState` , and one `OrderItem` per cart line — each a snapshot of that product's id/name/vendor/icon/price/quantity/fulfillment at that moment) and inserts it at the front of the shared, mutable `customerOrders` list (`models/marketplace_order.dart`). The cart and fulfillment maps are then cleared and a "Payment successful! Your order has been placed." snackbar shows.
4. Because `customerOrders` is a single global list (not scoped per-customer), it is also the *exact* data source `OrderHistoryScreen` reads, and — filtered by vendor name via `vendorOrderEntries()` — the exact data every vendor screen reads. There is no separate "vendor's copy" of an order; it's the same object.

Product Detail (`marketplace_product_detail_screen.dart`)

Hero photo (tap → `MarketplaceProductGalleryScreen`) plus a thumbnail strip if there's more than one photo, product name, vendor name, star rating, an in-stock/out-of-stock pill (" `N` left in stock" / "Out of stock", driven by `product.stock`), price, a **Quantity** stepper (–/+, floor 1, capped at `product.stock` ; hidden entirely when out of stock), a Description block, and an **Add to Cart** button. The button tracks the same cart state as the home card: **Add to Cart** (accent color) normally, **Already in Cart** (green) once the stepper's quantity matches what's already in the cart for this product, and disabled/relabeled **Out of Stock** when `stock == 0` . It opens pre-loaded with the stepper set to however many are already in the cart (or 1 if none).

Product Gallery (`widgets/marketplace_product_gallery_screen.dart`)

Same full-screen swipeable/zoomable viewer pattern as the property gallery, but takes `ImageProvider` s directly since a vendor-listed product's photos may be locally-picked files rather than bundled assets.

Order History (`order_history_screen.dart`)

Title "Order History". Every entry in `customerOrders` , newest first, each card showing the order date, payment method, every line item (name × quantity, vendor · fulfillment method, subtotal), the order **Total**, and — critically — a **Message [Vendor Name]** button *for each distinct vendor in that order the customer chose Pickup from* (`order.pickupVendors`). A delivery-only order shows no message button at all, since there's nothing to coordinate in person. Tapping it opens `ChatThreadScreen` pre-seeded with a message from the vendor: "Hi! Your order is ready whenever you'd like to come by for pickup." Empty state: "No orders yet — Things you buy on the Marketplace will show up here."

Marketplace Messages (`marketplace_messages_screen.dart`)

Title "Messages". Lists every distinct vendor name the customer has a pickup order with (derived the same way as Order History's message buttons — `order.pickupVendors` across all `customerOrders`), each opening the same seeded pickup-ready chat thread. If the customer has never bought a pickup item, the screen reads: "You can message a vendor once you've bought a pickup item from them."

---

7. Marketplace — Vendor Side

Vendor Login (`vendor_login_screen.dart`)

Title "Vendor Login — Sign in to manage your shop, products and orders." Email + password fields (password has a show/hide toggle). **Login** only checks that both fields are non-empty (shows a snackbar "Enter your email and password to continue" otherwise) — any values pass, replacing the current route with `VendorDashboardScreen` .

Vendor Sign Up (`vendor_signup_screen.dart`)

Title "Become a Vendor — Tell us about your business." Fields/controls, in order:

- **Add a Business Logo (optional)** — a circular tap target opening the shared upload picker (Camera Roll or Files).
- **Business Name**, **Owner's Full Name**, **Email Address**, **Phone Number** (plain text fields).
- **Business Category** — dropdown-style tile opening a bottom sheet: Furniture, Home Appliances, Electronics, Fittings & Fixtures, Décor, Tools & Equipment, Other.
- **Business Address** (multi-line).
- **State** — bottom sheet of all Nigerian states.
- **CAC/RC Number (optional)**.
- **Password** / **Confirm Password** (each with its own show/hide eye icon).
- A checkbox: **"I agree to the Home Servant Vendor Terms & Conditions"**.
- **Sign Up as a Vendor** button — validates that business name, owner name, email, phone, category, address, state, and password are all filled and the terms checkbox is checked (snackbar "Please fill in all required fields and accept the vendor terms" otherwise), and that password/confirm match (snackbar "Passwords do not match" otherwise). On success, replaces the route with `VendorSignupSuccessScreen` , passing the typed business name through — note this does **not** actually create or log into a new vendor account; the app's one mock vendor identity (`mockLoggedInVendor` = "Comfort Home Furniture") is unaffected.

Vendor Sign Up Success (`vendor_signup_success_screen.dart`)

A checkmark icon, "Application Submitted!", and body text: "Thanks for signing up, `<businessName>`. We'll review your vendor application and notify you once it's approved — this usually takes 24–48 hours." Button: **Back to Marketplace** — pops all the way back to the first route on the stack (i.e., back out of the Marketplace flow entirely, not into a vendor dashboard).

Vendor Dashboard (`vendor_dashboard_screen.dart`)

The vendor's home screen after logging in, built around `mockLoggedInVendor` (always "Comfort Home Furniture" in this build) and `vendorOrderEntries(vendor.businessName)`:

- Header: "Welcome back, `<businessName>`", a notification bell with an unread-count badge (count of order items where `notificationRead == false`) → `VendorNotificationsScreen`, a chat bubble icon → `VendorMessagesScreen`, and a logo/avatar circle.
- Three **stat cards**: **Products** (count of `marketplaceCatalog` entries with this vendor's name), **Orders** (count of `vendorOrderEntries`), **Revenue** (sum of every non-cancelled item's subtotal, formatted "₦…").
- **Recent Orders** list — every order item sold by this vendor, newest first, each tile showing product name, customer name + date, subtotal, and a status pill (**Pending** amber / **Completed** green / **Cancelled** red). Tapping a tile marks that item's notification as read and opens `VendorOrderDetailScreen`.
- Bottom nav (`VendorBottomNav`, 3 icons: dashboard/inventory/storefront) switches (via `pushReplacement`) between this screen, `VendorProductsScreen`, and `VendorProfileScreen`.

Vendor Products (`vendor_products_screen.dart`)

Title "My Products" with an **Add** button → pushes `AddProductScreen`; on a successful add, the list refreshes. Lists every `marketplaceCatalog` product belonging to this vendor (photo, name, category, price) with a per-row delete icon that removes it straight out of `marketplaceCatalog` — since the customer marketplace reads that same list, a removed product disappears from customer view immediately. Empty state: "You haven't listed any products yet."

Add Product (`add_product_screen.dart`)

Title "List a Product". Fields, in order:

- **Product Name**, **Description** (multi-line).
- **Photos (2-5) · N/5** — a horizontal strip of picked thumbnails (each removable via an × badge) plus an "add photo" tile; uses `pickMultipleImageUploads`.
- **Video (optional)** — a single optional video pick, shown as a filename chip once picked (removable).
- **Price** (₦, thousands-separated as you type) and **Quantity in Stock** (digits only).
- **Fulfillment** — an info box explicitly warning: "*List this item accurately as delivery only, pickup only, or both — customers can only choose from what you select here, so make sure it matches what you can actually fulfil.*" Below it, two checkbox-style tiles for **Delivery** and **Pickup** (`FulfillmentMethod` values) — at least one must be selected.
- **List Product** button validates: name & description non-empty; at least 2 photos; a positive price; a non-negative stock number; at least one fulfillment method selected — showing a specific snackbar for whichever check fails. On success it appends a new `MarketplaceProduct` (category taken from the vendor's own `category`, rating starting at 0) to the shared `marketplaceCatalog` and pops back with `true`.

Vendor Profile / "Shop Profile" (`vendor_profile_screen.dart`)

Title "Shop Profile" with an **Edit Profile** pill button (top-right) → `VendorEditProfileScreen`; returning refreshes the page. Shows the shop logo, business name, and star rating, then a details card (**Owner**, **Email**, **Category**, **State**). Below that:

- **Transaction History** row → `VendorTransactionsScreen`.
- **Contact Support** row → opens the same shared support-options sheet (Call Support / Live Chat) used on the tenant Settings screen.
- **Log Out** row → pops all the way back to the first route on the stack (leaves the vendor area entirely, same as the customer-side "Back to Marketplace").
- **DANGER ZONE**: **Deactivate Shop** → confirmation sheet ("Your products will be taken off the marketplace and customers won't be able to reach you. You can become a vendor again any time.") → shows a "Your shop has been deactivated" snackbar and pops to the first route. Note: this does not actually remove the vendor's products from `marketplaceCatalog` in this build — it's a session-exit action, not a data mutation.

Vendor Edit Profile (`vendor_edit_profile_screen.dart`)

Title "Edit Shop Profile". Logo picker (camera-badge avatar), **Business Name**, **Owner's Full Name**, **Email Address**, **Business Category** (bottom-sheet picker). Then a **Payout Account** section, preceded by this exact disclaimer text in an info box:

> "Payouts are sent to this account only after an order has been verified and marked completed — not immediately at purchase."

Followed by **Bank Name**, **Account Number**, **Account Name** fields (all optional — blank clears the stored value). **Save Changes** requires a non-empty business name (snackbar "Business name cannot be empty" otherwise); if the business name actually changed, it cascades the rename across every product in `marketplaceCatalog` and every historical `OrderItem` in `customerOrders` that carried the old name, so "My Products" and every order/notification/transaction screen keep matching correctly. Shows "Shop profile updated" and pops.

Vendor Notifications (`vendor_notifications_screen.dart`)

Title "Notifications". One row per `VendorOrderEntry` for this vendor (from `vendorOrderEntries`), newest first: "New order · `<product name>`", customer name · fulfillment · subtotal, an unread dot, and a status pill. Tapping marks it read and opens `VendorOrderDetailScreen`. Empty state: "Orders for your shop will show up here."

Vendor Order Detail (`vendor_order_detail_screen.dart`)

Shared detail screen reached from both Notifications and Recent Orders/Transaction History. Shows the item (icon, name, qty × unit price, status pill), an order-info card (Order ID, Order Date, Payment Method, Fulfillment, Item Total), and a **Customer** card whose contents *depend on fulfillment method*:

- **Delivery** items show the customer's **Phone** and **Delivery Address** (or "Not provided" if blank).
- **Pickup** items instead show only "Fulfillment: Customer will pick up" — no phone/address shown — plus a **Message Customer** outline button that opens `ChatThreadScreen` seeded with the vendor asking when the customer would like to pick up.

If the item's status is still **Pending**, two buttons appear: **Cancel Order** (outlined red, sets status to `cancelled`) and **Mark Completed** (filled, sets status to `completed`) — both call `_setStatus`, which mutates the shared `OrderItem.status` in place (so the change is immediately visible from the customer's Order History too, since it's the same object) and shows a "Marked as `<Status>`" snackbar. Once an item is Completed or Cancelled, these buttons no longer appear (a status change here is one-way in this prototype).

Vendor Transactions (`vendor_transactions_screen.dart`)

Title "Transaction History". A summary card: **"Total from completed orders"** → ~~₦~~`<sum of all Completed items' subtotals>`. Below it, every `VendorOrderEntry` for this vendor as a row (product name, customer name · payment method, subtotal, status pill) — tapping one opens the same `VendorOrderDetailScreen`. Empty state: "Your sales will show up here once you get an order."

Vendor Messages (`vendor_messages_screen.dart`)

Title "Messages". Lists every pickup order this vendor has (`vendorOrderEntries(...).where(fulfillment == pickup)`), each opening a chat thread with that customer, seeded with the vendor's pickup-scheduling question. Empty state: "You can message a customer once they've bought a pickup item from you."

The architectural key fact

`models/marketplace_order.dart` defines `vendorOrderEntries(String vendorName)`, which filters the single global `customerOrders` list down to `VendorOrderEntry(order, item)` pairs where `item.vendorName == vendorName`. **Every vendor-side screen (Dashboard's Recent Orders, Notifications, Transactions, Messages) and the customer-side Order History all read from this one shared list** — there is no separate "vendor's mock orders" dataset. A status change made in `VendorOrderDetailScreen`, or a business-name rename made in `VendorEditProfileScreen`, is instantly visible everywhere else that reads the same objects, because Dart lists/objects are shared by reference in memory (again, none of this is persisted to a real backend — it resets on a full app/data reload since `customerOrders` and `marketplaceCatalog` are plain in-memory globals, not saved via `AppState`'s `SharedPreferences` persistence).

---

8. Cross-Cutting Features

App Lock (`widgets/app_lock_gate.dart` , `app_lock_screen.dart` , `app_lock_pin_sheet.dart`)

- **Setup**: toggling "App Lock" on in Settings calls `showSetAppLockPinSheet`, a two-step "enter, then confirm" 4-digit PIN flow (`_SetPinSheet`) — if the confirmation doesn't match the first entry, it shows "PINs didn't match — try again" and restarts both steps. The confirmed PIN is stored via `AppState.enableAppLock(pin)`.
- **Turning off**: requires re-entering the existing PIN via `showVerifyAppLockPinSheet` (title "Enter your PIN to turn off App Lock"); a wrong PIN shows "Incorrect PIN" and clears the entry without disabling anything. A **Cancel** button is available.
- **Enforcement**: `AppLockGate` wraps the entire app (via `MaterialApp.builder`) and is a `WidgetsBindingObserver`. It shows a full-screen, non-dismissible (`PopScope(canPop: false)`) `AppLockScreen` whenever the app resumes from being backgrounded (paused/hidden → resumed) while App Lock is enabled, and also on a cold start if App Lock was already on *before* this launch (checked once, from the value `AppState.isLoaded` restores — turning App Lock on mid-session doesn't immediately re-lock the session you're already in). `AppLockScreen` itself is a 4-digit keypad (`PinKeypad` / `PinDots`) that calls `onUnlocked` the instant the entered digits match, or shows "Incorrect PIN" and clears the entry on a mismatch.
- The go_router `/app-lock-verify` route uses the same `AppLockScreen` as a post-login gate (see Navigation Map).

Support Sheet (`widgets/support_sheet.dart`)

`showSupportOptionsSheet(context, theme: ...)` — a bottom sheet titled "Contact Support" with two rows:

- **Call Support** (subtitle shows the literal phone number `+234 700 000 0000`) — attempts to launch the device dialer via `url_launcher`; if that fails, shows a snackbar with the number to dial manually.
- **Live Chat** (subtitle "Chat with our support team") — opens `ChatThreadScreen` seeded with "Hi! How can we help today?" from "HomeServant Support".

Used from two places: the tenant/landlord **Settings → Help & Support** row, and the vendor **Shop Profile → Contact Support** row — both get the exact same scripted support thread.

Theming (`models/dashboard_theme.dart` , `widgets/theme_picker_sheet.dart`)

`DashboardTheme` has three values, each recombining the same three brand colors (navy, sand `#F2CF8F`, white) into different roles (`background`, `surface`, `accent`, `foreground`, `onSurface`, `onAccent`, `navigatorColor`, etc.):

- **Midnight** — navy background, white/sand accents (dark mode-like).
- **Sand** — sand background, navy accents.
- **Classic White** — white background, sand surface, navy accents (the default).

Picked from the Profile screen's **Theme** row via `showThemePickerSheet` ("Choose a Theme", one tile per option with a 3-dot color swatch preview and a checkmark on the current selection). Selecting a theme calls `AppState.setDashboardTheme`, which also fires `AppIconService.apply(theme)` — a best-effort attempt to switch the OS home-screen app icon via a native platform channel (`com.homeservant/app_icon`), silently ignored wherever that channel isn't wired up (web/desktop, or if the native side doesn't support it).

Upload Picker (`widgets/upload_picker.dart`)

Two entry points, both offering the same bottom-sheet choice of **Choose from Camera Roll** or **Choose from Files**:

- `pickUpload(context)` — single file (image or any document type), used for profile photos, landlord's Certificate of Ownership, ID verification documents, and vendor/product logos.
- `pickMultipleImageUploads(context, maxCount: n)` — multiple images at once (images only), used specifically by the Add Product screen's 2–5 photo requirement.

Both return `PickedUpload` object(s) carrying a file path, filename, an `isImage` flag, and (on web, where a real file path isn't available for "Choose from Files") raw bytes — `PickedUpload.imageProvider` picks whichever of bytes/path is usable to actually render the picked image.

---

9. Data Models Glossary

All of the following are **in-memory only** unless noted — they reset on a full app reload/restart except where `AppState` explicitly persists via `SharedPreferences`.

Model	File	Represents	Persistence
`AppState`	`lib/state/app_state.dart`	The entire signed-in session: role, profile fields (name/email/phone/address/DOB/photo), wishlist (`favoritePropertyIds`), rental history (`_rentalHistory`), notification toggles, 2FA/App-Lock settings, dashboard theme, referral code.	**Persisted** locally via `SharedPr` on every change (`notifyListe` a save); survives app restart but the device.
`Property`	`lib/features/dashboard/models/property.dart`	One rental listing (title, location/state, rating, image, category, price/priceUnit, bed/bath count, description, landlord name, gallery images).	In-memory constant list `mockPr` seeded listings) — never mutatec
`RentalRecord`	`lib/features/dashboard/models/rental_record.dart`	A tenant's active/past rent or shortlet booking for one property (start/end date, optional 1–5 star rating). Keyed by property id inside `AppState`.	Persisted as part of `AppState`
`MarketplaceProduct`	`lib/features/Market place/models/marketplace_product.dart`	One item for sale on the Marketplace (name, vendor, category, price, rating, stock, photos/video, supported fulfillment methods).	In-memory mutable global list `marketplaceCatalog` (seed `mockMarketplaceProducts` to/removed-from live by vendor a resets on reload.
`MarketplaceOrder` / `OrderItem`	`lib/features/Market place/models/marketplace_order.dart`	A completed purchase (`MarketplaceOrder`: id, date, items, payment method, customer contact snapshot) and its line items (`OrderItem`: product snapshot, vendor name, quantity, fulfillment, mutable status, notification-read flag). `vendorOrderEntries(vendorName)` is the shared query every vendor screen uses to filter these by seller.	In-memory mutable global list `customerOrders` (seeded wit orders, then appended-to at che on reload. This is the single sourc shared by the customer's Order H every vendor's Dashboard/Notifications/Transac screen.
`FulfillmentMethod`, `PaymentMethod`, `OrderItemStatus`	`lib/features/Market place/models/order_options.dart`	Enums: `FulfillmentMethod` (delivery/pickup), `PaymentMethod` (card/bankTransfer/payOnDelivery), `OrderItemStatus` (pending/completed/cancelled, each with a label + status color).	N/A (pure enums).
`VendorProfile`	`lib/features/Market place/models/vendor.dart`	The signed-in vendor's shop profile (business name, owner, email, category, state, rating, logo path, optional payout bank details).	Single mutable global `mockLoggedInVendor` ("Com Furniture") — there is exactly one identity in the whole app; edits m and reset on reload.
`UserRole`	`lib/models/user_role.dart`	Tenant vs. Landlord, plus the navy/gold color scheme each uses on auth/onboarding screens.	N/A (enum), current value stored `AppState.role` (persisted).
`DashboardTheme`	`lib/models/dashboard_theme.dart`	The 3 selectable post-login color themes (Midnight/Sand/Classic White) and every derived color role used by dashboard/marketplace/vendor screens.	N/A (enum), current value stored `AppState.dashboardTheme`

Coverage Note

Every screen/widget/model file listed in the task brief was opened and read directly from source, including: `lib/features/splash/splash_screen.dart`; `lib/features/onboarding/get_started_screen.dart`; every file in `lib/features/auth/`; every file in `lib/features/dashboard/` (screens, `models/`, `widgets/`); `lib/features/landlord/landlord_dashboard_screen.dart`; every file in `lib/features/Market place/` (customer screens, vendor screens, `models/`, `widgets/`); every shared widget in `lib/widgets/`; `lib/models/user_role.dart` and `lib/models/dashboard_theme.dart`; `lib/state/app_state.dart`; `lib/routes/app_router.dart`; `lib/app.dart` / `lib/main.dart`; and the supporting `lib/core/` theme/responsive/formatting files and `lib/services/app_icon_service.dart`.

The only file in `lib/` not covered above is `lib/main_icon_debug.dart`, a standalone debug entry point (a separate `main()` for previewing app icon assets) that is not part of the user-facing screen flow and has no bearing on app functionality described in this document.